

Day 2 – Standalone Components & Templates

Setup & Prerequisites

1. Setup Checklist:
- **Virtual Environment/Dev Server:** Ensure your terminal is in the project folder and you run `ng serve`.
 - **IDE Ready:** Open VS Code and open `src/app/app.component.html` and `src/app/app.component.ts`.
2. Prerequisites:
- A scaffolded Angular workspace from Day 1 with `ng serve` verified.

Learning Objectives

- By the end of this class, you will be able to:
- Generate standalone components using the Angular CLI.
 - Explain the role of the `@Component` decorator and metadata fields.
 - Use interpolation and property binding to pass data from TypeScript to HTML.
 - Use event binding to trigger component methods from the HTML template.
 - Implement two-way data binding using `[(ngModel)]`.
 - Apply Angular 22 LTS control flow syntax (`@if`, `@else`, and `@for`) for dynamic rendering.

Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Visualizing component hierarchy. Understanding Standalone components vs Modules.
2. Key Concepts & Core Ideas	7 min	Defining decorators, property/event bindings, two-way bindings, and modern control flow.
3. Live Coding Walkthrough	23 min	Generate components, implement logic and layouts, handle user input, and display collections.

4. Check for Understanding	5 min	Q&A on binding directionality, control flow syntax, and template variables.
5. Class Wrap-up & Checkpoint	3 min	Verify interactive features run correctly in the browser.

Step-by-Step Walkthrough

1. Hook & Big Picture (7 min)

Websites are made of reusable visual blocks. In Angular, we call these blocks **Components**. Every Angular application is a tree of components, starting with the root `AppComponent`. Standalone components do not need external containers called modules; they are self-declaring, meaning each component explicitly declares what dependencies it needs to compile.

- **Component Tree:** The root component (`AppComponent`) holds the main layout. We will generate child components (like `GameCardComponent` or `GameListComponent`) that fit inside it, passing data down and bubble events up.

2. Key Concepts & Core Ideas (7 min)

Introduce these template and logic concepts:

- **@Component Decorator:** A TypeScript decorator that attaches configuration metadata to a class, turning it into an Angular component. Important fields include:
 - `selector`: The custom HTML tag name (e.g. `<app-game-list>`).
 - `imports`: Array of standalone components, directives, or pipes this component uses.
 - `templateUrl`: Path to the HTML layout file.
 - `styleUrls`: Path to the CSS styling files.
- **Interpolation (`{{ value }}`):** Renders dynamic string values from the TypeScript class inside the HTML template.
- **Property Binding (`[property]="value"`):** Passes values one-way from the component class to HTML element attributes (e.g. setting `[disabled]` or `[src]`).
- **Event Binding (`(event)="method()"`):** Captures user interactions from the browser and invokes methods in the component class (e.g. `(click)`).
- **Two-Way Binding (`[(ngModel)]`):** Synchronizes inputs in real-time. Changes in the input box update the TypeScript variable, and changes in TypeScript update the input box.
- **Modern Control Flow (`@if`, `@for`):** Built-in template blocks for conditional rendering and looping through lists without needing legacy directives like `*ngIf` or `*ngFor`.

3. Live Coding Walkthrough (23 min)

Step 3.1: Creating a Standalone Component

- **Concept:** Generate a new standalone component called `game-list` using the Angular CLI.

- **Code:**

```
# Generate a standalone component under src/app/components/game-list/  
ng generate component components/game-list
```

- **Verify:** Confirm that the folder `src/app/components/game-list/` is created containing `.ts`, `.html`, `.css`, and `.spec.ts` (test) files.

Step 3.2: Implementing Component Data Logic

- **Concept:** Open `src/app/components/game-list/game-list.component.ts`. We will define a mock collection of games, a new game text box input, and methods to add or remove games.
- **Code:** Modify `src/app/components/game-list/game-list.component.ts`:

```
import { Component } from '@angular/core';  
import { CommonModule } from '@angular/common';  
import { FormsModule } from '@angular/forms'; // Required for [(ngModel)]  
  
// Definition of a TypeScript interface representing a Game object  
interface Game {  
  id: number;  
  title: string;  
  price: number;  
  available: boolean;  
}  
  
@Component({  
  selector: 'app-game-list',  
  standalone: true,  
  // We import CommonModule for general utilities, and FormsModule for [(ngModel)]  
  imports: [CommonModule, FormsModule],  
  templateUrl: './game-list.component.html',  
  styleUrls: ['./game-list.component.css']  
})  
export class GameListComponent {  
  // Component state properties  
  newGameTitle: string = '';  
  newGamePrice: number = 0;  
  
  games: Game[] = [  
    { id: 1, title: 'Half-Life 3', price: 29.99, available: true },  
    { id: 2, title: 'Cyberpunk 2077', price: 49.99, available: false },  
    { id: 3, title: 'Portal 3', price: 19.99, available: true }  
  ];  
};
```

```

// Method to handle adding a game to the array list
addGame(): void {
  if (this.newGameTitle.trim() === '') return;

  const newGame: Game = {
    id: this.games.length + 1,
    title: this.newGameTitle,
    price: this.newGamePrice,
    available: true
  };

  this.games.push(newGame);
  // Reset input form boxes
  this.newGameTitle = '';
  this.newGamePrice = 0;
}

// Method to toggle availability of a game
toggleAvailability(game: Game): void {
  game.available = !game.available;
}
}

```

Step 3.3: Creating the Dynamic HTML Template

- **Concept:** Open `src/app/components/game-list/game-list.component.html`. We will create the UI containing inputs for adding a game, and lists to display current games. We will use the new `@if` and `@for` control flow blocks.
- **Code:** Modify `src/app/components/game-list/game-list.component.html`:

```

<div class="container">
  <h2>🎮 Game Inventory Management</h2>

  <!-- Add Game Input Form Section -->
  <div class="form-group">
    <input type="text" [(ngModel)]="newGameTitle" placeholder="Enter game title" class="input-
    <input type="number" [(ngModel)]="newGamePrice" placeholder="Price" class="input-box" />
    <!-- (click) event binding calls the addGame() component class method -->
    <button (click)="addGame()" class="btn-add">Add Game</button>
  </div>

  <!-- Conditional rendering with @if. If games list is empty, show alert -->
  @if (games.length === 0) {
    <p class="empty-msg">No games in inventory.</p>
  } @else {
    <ul class="game-items">
      <!-- Looping with @for. track determines the identifier to optimize DOM updates -->
      @for (game of games; track game.id) {
        <li class="game-item">
          <!-- Interpolation displays values inside the template -->
          <span class="title">{{ game.title }}</span>

```

```

        <span class="price">${{ game.price | number:'1.2-2' }}</span>

        <!-- Conditional styling and labels using @if -->
        @if (game.available) {
            <span class="status available">In Stock</span>
        } @else {
            <span class="status out-of-stock">Out of Stock</span>
        }

        <!-- Event binding triggers state changes -->
        <button (click)="toggleAvailability(game)" class="btn-toggle">
            Toggle Availability
        </button>
    </li>
}
</ul>
}
</div>

```

Step 3.4: Bootstrapping into the Root Layout

- **Concept:** To display the new component, we must register it inside the root AppComponent imports array and insert its HTML selector tag in the root template.
- **Code:** Modify `src/app/app.component.ts`:

```

import { Component } from '@angular/core';
import { GameListComponent } from '../components/game-list/game-list.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [GameListComponent], // Import custom component
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'gamekey-store';
}

```

Modify `src/app/app.component.html` (delete the default boilerplate and add the selector):

```

<main>
  <h1>Welcome to GameKey Store</h1>
  <!-- Custom HTML element selector tag of our GameListComponent -->
  <app-game-list></app-game-list>
</main>

```

- **Verify:** Check your browser at `http://localhost:4200/`. Confirm that the game list renders. Type a new title and price, click "Add Game", and verify it is appended to the list. Click "Toggle Availability" and

check that the status label updates instantly.

4. Check for Understanding (5 min)

Question: "What is the purpose of the `track` keyword in the new `@for` template syntax?"

Answer: `track` is used by Angular's rendering engine to uniquely identify list items. If the collection changes (e.g. an item is added, moved, or deleted), Angular uses the track key (like `game.id`) to update only the modified DOM node, rather than destroying and rebuilding the entire list. This improves UI performance.

Question: "What's the difference between property binding `[disabled]='val'` and string interpolation `disabled='{{val}}'`?"

Answer: Property binding passes raw data values (booleans, numbers, objects) directly to the element's DOM property. String interpolation converts everything to a string before assigning it. For boolean attributes like `disabled`, passing a string `"false"` counts as true, so property binding is required.

Question: "Why do we need to import `FormsModule` in standalone component metadata imports to use `[(ngModel)]`?"

Answer: `[(ngModel)]` is a custom directive defined inside Angular's `FormsModule` package. Since standalone components explicitly manage their own imports, they must import `FormsModule` directly to tell the compiler how to compile two-way data-binding logic.

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria:**

- ☐ The `GameListComponent` is generated successfully as a standalone component.
- ☐ Adding a game via input boxes updates the browser display list in real-time.
- ☐ The `@if` and `@for` control flow blocks render conditional elements correctly.